OOP Project Reflection Report

Reinforcement Learning with Object-Oriented Design

repo link: https://github.com/TWCkaijin/OOP-project

B123245005 吳楷鈞, B123245009 黃皓群, B123245017 王心妤

1. Project Overview

This project applies Reinforcement Learning techniques while putting Object-Oriented Programming principles into practice. The work is divided into three parts:

- **Part 1**: Solving the Mountain Car problem using Q-Learning
- **Part 2**: Navigating Frozen Lake 8x8 using SARSA
- **Part 3**: Building a custom Warehouse Robot environment

Part 3 serves as the main focus for demonstrating OOP concepts. The environment simulates a warehouse where a robot navigates through obstacles, picks up cargo scattered across the grid, and returns to a delivery point. The robot can only carry a limited number of items at once, requiring multiple trips to complete the task.

2. Team Contributions

Reinforcement Learning (Part 1 & 2)

- Q-Learning with discretized state space for Mountain Car
- SARSA implementation for Frozen Lake navigation

Warehouse Robot Environment (Part 3)

- Custom Gymnasium environment with three curriculum stages
- Pygame visualization with sprite graphics
- DQN/PPO training via Stable-Baselines3 with checkpoint support
- Greedy AI opponent and random obstacle generation
- OOP refactoring using Strategy Pattern

Infrastructure: GitHub Actions for automated training and evaluation

3. OOP Concepts Applied

Abstract Classes and Inheritance

We used Python's `abc` module to define abstract base classes. This ensures that any new strategy class must implement the required interface before it can be used:

```
class ObstacleGenerator(ABC):
    @abstractmethod
    def generate(self, safe_zones: list) -> list:
        pass
```

```
class FixedObstacleGenerator(ObstacleGenerator): ...
class RandomObstacleGenerator(ObstacleGenerator): ...
```

The environment code can then use any generator through the common interface without knowing which specific implementation is being used.

Strategy Pattern

Three modules follow the Strategy Pattern, each providing multiple interchangeable implementations:

Module	Available Strategies
`obstacles.py`	Fixed, Random, Empty
`rewards.py`	Basic, Shaping, Competitive
`robots.py`	Greedy, Random, Patrol

This design makes adding new strategies straightforward without modifying existing code.

Multi-level Inheritance

The reward strategies demonstrate inheritance chains. `CompetitiveReward` extends `ShapingReward` rather than starting from scratch, reusing the parent's distance-shaping logic and adding competition-specific bonuses on top via `super()`.

4. Challenges and Solutions

Challenge	How We Handled It
Preserving existing functionality during refactoring	Made small, incremental changes and ran tests after each modification to catch regressions early
Keeping new robot classes in sync with rendering	Updated position state before each decision call to ensure consistency
Maintaining consistent code style across contributors	Studied existing naming patterns and comment conventions, then followed the same style

5. What We Learned

- **Strategy Pattern**: Makes code flexible without complexity. Switching strategies requires one line change.
- **Value of Abstraction**: Well-defined interfaces allow new implementations without modifying other code.
- **Refactoring Discipline**: Changing structure while keeping behavior requires careful testing.
- **Tooling**: Gained experience with Python's `abc` module, Gymnasium API, and Git workflows.

6. Team Collaboration

The team used Git with feature branches and Conventional Commits format. GitHub Actions handled automated training and evaluations.

7. Conclusion

This project was a practical opportunity to see how OOP principles work in a real codebase rather than textbook examples. The Strategy Pattern turned out to be genuinely useful for managing different behaviors, not just an academic exercise. We came away with a better sense of when abstraction helps improve code quality and when it might add unnecessary complexity.
